

University of Botswana

Department of Computer Science

CSI481 Database Systems

Assignment Part 2: Logical Design and Database Implementation

Infant Immunisation Recording System

Submission date: 23 October 2026

Name	Student ID
[Member 1 name]	[ID]
[Member 2 name]	[ID]
[Member 3 name]	[ID]
[Member 4 name]	[ID]
[Member 5 name]	[ID]

1. Introduction

This document presents the logical database design for the Infant Immunisation Recording System, translated from the model solution conceptual class diagram supplied for this assignment. It sets out the relational schema implemented in MySQL/MariaDB, the reasoning behind every decision that the conceptual diagram itself left open, the integrity constraints enforced at the database level, and the correspondence between each application requirement and the tables and queries that satisfy it.

This translation deliberately follows the model solution diagram supplied for the assignment rather than the group's own Part 1 conceptual design, since the brief for Part 2 explicitly requires working from that model solution. Two of the classes in that diagram, District and Practice, carry no marked identifying attribute at all, and two more classes, Vaccine and Vaccination, are in the same position. Where the diagram is silent on identity, a surrogate key is introduced rather than an attribute being invented that the diagram does not actually contain. This is stated plainly here so the reasoning is auditable rather than assumed.

2. From Conceptual Model to Logical Schema

2.1 Classes and Identifiers

Seven classes appear on the diagram: District, Practice, GP, Guardian, Patient, Vaccine and Vaccination. Each becomes one relation. The table below records, for each class, what the diagram gives as an identifying attribute and what was used as the primary key in the schema actually implemented.

Class	Diagram Identifier	Primary Key Used	Reason
District	none given	DistrictID (surrogate)	No attribute on the diagram distinguishes

			districts; Name is treated as a candidate unique attribute instead
Practice	none given	PracticeID (surrogate)	Diagram gives only Location; no naming attribute is present despite the scenario text referring to practice names
GP	GPN0	GPN0 (natural)	Directly stated as the identifying attribute in both the scenario and the diagram
Guardian	ON0 (Omang)	ON0 (natural)	Directly stated as the identifying attribute
Patient	BirthCertNo	BirthCertNo (natural)	Directly stated as the identifying attribute, per R2 of the scenario
Vaccine	none given	VaccineID (surrogate)	No attribute on the diagram distinguishes vaccines
Vaccination	none given	VaccinationID (surrogate)	An event class with no natural identifier; identity is really the combination of patient, vaccine and booster number, enforced separately as a uniqueness constraint (Section 5, C2)

2.2 Multiplicities and Foreign Key Nullability

Every association's multiplicities on the diagram were read directly into whether the corresponding foreign key is declared NOT NULL. A lower bound of 1 on an association end means the class at the other end cannot exist without that link, which is enforced as a mandatory foreign key. A lower bound of 0 means the link may legitimately be absent for a time, which the schema does not need to enforce as mandatory.

Association (from diagram)	Reading	Resulting Constraint
District 1 -- 1..* Practice	Every practice belongs to exactly one district	Practice.DistrictID NOT NULL
Practice 1 -- 1..* GP	Every GP works at exactly one practice	GP.PracticeID NOT NULL
GP 1 -- 1..* Patient (CaresFor)	Every patient is on exactly one GP's list	Patient.GPN0 NOT NULL
Guardian 1 -- 1..* Patient	Every patient has exactly one guardian	Patient.ON0 NOT NULL
GP 1 -- 1..* Vaccination	Every administered vaccination	Vaccination.GPN0 NOT

	has exactly one administering GP	NULL
Patient 1 -- 1..* Vaccination	Every vaccination belongs to exactly one patient	Vaccination.BirthCertNo NOT NULL
Vaccine 1 -- 1..* Vaccination (GivenAs)	Every vaccination is for exactly one vaccine	Vaccination.VaccineID NOT NULL

Every foreign key in the schema is therefore NOT NULL, with no exceptions. This is not an incidental choice: it is the direct consequence of every association on the diagram being drawn with a mandatory (1, not 0..1) lower bound on the "one" side. Where an insert cannot supply one of these values, for example adding a GP before any practice exists, the correct response is to create the missing parent record first (which the application's quick-add dialogs support inline), not to relax the constraint.

3. Relational Schema

Full DDL is provided in schema.sql, accompanying this document and the submitted executable. The definition of each relation is reproduced here for reference.

District

```
District (
  DistrictID INT AUTO_INCREMENT PRIMARY KEY,
  Name      VARCHAR(100) NOT NULL UNIQUE
)
```

Practice

```
Practice (
  PracticeID INT AUTO_INCREMENT PRIMARY KEY,
  Location   VARCHAR(150) NOT NULL,
  DistrictID INT NOT NULL REFERENCES District(DistrictID)
)
```

GP

```
GP (
  GPNo      INT PRIMARY KEY,
  Name      VARCHAR(100) NOT NULL,
  PracticeID INT NOT NULL REFERENCES Practice(PracticeID)
)
-- /NoOfPatients is derived (Section 5, C-derived); computed via
-- COUNT(*) over Patient, never stored as a column here.
```

Guardian

```
Guardian (
  ONo INT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL
)
```

Patient

```
Patient (
  BirthCertNo INT PRIMARY KEY,
  Name        VARCHAR(100) NOT NULL,
```

```

DateOfBirth DATE NOT NULL,
ONo          INT NOT NULL REFERENCES Guardian(ONo),
GPNo         INT NOT NULL REFERENCES GP(GPNo)
)

```

Vaccine

```

Vaccine (
  VaccineID  INT AUTO_INCREMENT PRIMARY KEY,
  Name       VARCHAR(100) NOT NULL,
  Manufacturer VARCHAR(100),
  Type       VARCHAR(50)
)

```

Vaccination (doses already administered)

```

Vaccination (
  VaccinationID INT AUTO_INCREMENT PRIMARY KEY,
  DateOfVac     DATE NOT NULL,
  BoosterNo     INT NOT NULL DEFAULT 1,
  BatchNo       VARCHAR(50) NOT NULL,
  BirthCertNo   INT NOT NULL REFERENCES Patient(BirthCertNo),
  GPNo          INT NOT NULL REFERENCES GP(GPNo),
  VaccineID     INT NOT NULL REFERENCES Vaccine(VaccineID),
  UNIQUE (BirthCertNo, VaccineID, BoosterNo)
)

```

4. A Necessary Addition: the Appointment Relation

The model solution diagram has no class representing a vaccination that has been scheduled but not yet given. Its only event class, Vaccination, carries DateOfVac as a date of something that has already happened, and nothing else. Three separate requirements in the application brief cannot be met from that alone:

GUI task 3 requires the system to "schedule the next booster vaccination for a patient... and appoint with a GP for the scheduled vaccination." A schedule is inherently a future commitment, not a completed event.

The weekly work-list report requires listing what a practice needs to do in a given week, which by definition has not happened yet.

Appointment letters are, by their nature, letters informing a guardian of a future appointment, not a record of a past one.

Rather than overload Vaccination with a nullable status field and a date that sometimes means "given" and sometimes means "due," which would blur the very distinction constraint C2 relies on (Section 5), a second relation, Appointment, was added. It mirrors Vaccination's structure closely (same three foreign keys, a booster number) but adds a Status column (SCHEDULED, COMPLETED or CANCELLED) and stores a future ScheduledDate rather than a past DateOfVac. This keeps Vaccination exactly as the diagram specifies it: a clean historical record of doses actually given, and nothing else.

```

Appointment (
  AppointmentID INT AUTO_INCREMENT PRIMARY KEY,
  ScheduledDate DATE NOT NULL,
  BoosterNo     INT NOT NULL DEFAULT 1,
  BirthCertNo   INT NOT NULL REFERENCES Patient(BirthCertNo),

```

GPNo	INT NOT NULL REFERENCES GP(GPNo),
VaccineID	INT NOT NULL REFERENCES Vaccine(VaccineID),
Status	ENUM('SCHEDULED', 'COMPLETED', 'CANCELLED') NOT NULL DEFAULT 'SCHEDULED'
)	

This is disclosed here explicitly, rather than folded silently into the schema, because a deviation from a supplied model solution should always be visible and defensible, not discovered by a marker comparing the diagram to the tables line by line.

5. Business Rules and Integrity Constraints

C1. Uniqueness of identifiers. GPNo, ONo, BirthCertNo and VaccineID/DistrictID/PracticeID (surrogate) are each unique, enforced as primary keys. District.Name additionally carries a UNIQUE constraint, since the diagram's own text treats districts as named, distinguishable regions even though no identifying attribute was marked.

C2. One administered dose per booster number. A UNIQUE constraint on (BirthCertNo, VaccineID, BoosterNo) in Vaccination prevents the same patient being recorded as having received the same numbered dose of the same vaccine twice.

C3. Temporal validity. A vaccination cannot be dated before the patient's date of birth. This is enforced at the application layer (RegisterVaccinationPanel checks DateOfVac against Patient.DateOfBirth before inserting) rather than as a database CHECK constraint, since it requires comparing values across two different tables, which standard CHECK constraints in MySQL/MariaDB cannot reference.

C4. Booster numbers are positive integers. Enforced with CHECK (BoosterNo >= 1) on both Vaccination and Appointment.

C5. Derived attribute: GP.NoOfPatients. Never stored. Computed on demand as COUNT(*) grouped by GPNo over Patient (GPDAO.findAll()), which guarantees it can never drift out of sync with the actual set of registered patients, the same reasoning given for this attribute in the Part 1 report.

C6. Referential integrity on delete. All foreign keys use ON DELETE RESTRICT rather than CASCADE. Deleting a District, Practice, GP, Guardian, Patient or Vaccine that still has dependent rows is refused by the database rather than silently deleting the history that depends on it, which matters directly for an immunisation record whose entire purpose is not to lose history.

6. Storage Parameters and Implementation Choices

The schema targets MariaDB (MySQL-compatible) running under the InnoDB storage engine specifically, chosen over the default MyISAM alternative because InnoDB enforces foreign key constraints; MyISAM silently ignores them, which would make every constraint described in Section 5 unenforceable at the database level regardless of how it is declared.

Character set utf8mb4 with collation utf8mb4_unicode_ci is used throughout so that names are stored and compared correctly regardless of script, rather than defaulting to a Latin-only character set that would corrupt or reject non-ASCII names.

Two secondary indexes are created beyond the primary and foreign keys: one on Vaccination.DateOfVac and one on the composite (Appointment.ScheduledDate, Appointment.Status). Both correspond directly

to the WHERE clauses the reporting queries actually run: the six-month audit filters Vaccination by date range, and the weekly work-list filters Appointment by date range and status together. Without these indexes, both reports would force a full table scan as the number of vaccination records grows over the life of the system.

7. Test Data

seed.sql populates every relation with enough rows to exercise every screen and every report without further data entry: three districts, four practices, five GPs, five guardians, six patients, five vaccine types, fourteen administered vaccination records spread across the preceding six months (so the audit report returns real, non-trivial counts), and five scheduled appointments, one of them already cancelled, spanning the week immediately following the assignment's working date so the weekly work-list report has a populated week to display by default.

8. Mapping Application Requirements to the Schema

Requirement	Relations Used	Notes
Add GP / vaccine / patient / guardian	GP, Vaccine, Patient, Guardian, Practice, District	Foreign keys chosen from drop-downs only; quick-add dialogs insert a Practice or Guardian inline if needed
Register a vaccination	Vaccination, Patient, GP, Vaccine	All three foreign keys selected from existing records; no re-entry of their details
Schedule next booster + appoint GP	Appointment, Vaccination (read), Patient, GP, Vaccine	BoosterNo proposed as MAX(BoosterNo)+1 from Vaccination; interval is a hard-coded business rule, since no ScheduleEntry-equivalent exists on this diagram (see BoosterScheduler)
List patients and vaccination details	Vaccination JOIN Patient JOIN GP JOIN Vaccine	Single joined query, no unstated joins
Appointment letters	Appointment JOIN Patient JOIN Guardian JOIN GP JOIN Practice JOIN Vaccine	One appointment resolved to a full letter in one query
Weekly work-list by practice	Appointment JOIN GP (filter Practice + date range + Status)	Requires the Appointment addition described in Section 4
Six-month audit by vaccination type	Vaccination JOIN Vaccine, GROUP BY Type	Filtered on DateOfVac between today minus six months and today

References

Beynon-Davies, P. (2004). Database Systems, 3rd edition. Basingstoke: Palgrave Macmillan.

Connolly, T. and Begg, C. (Third Edition). Database Systems. Boston: Addison Wesley.

Oracle Corporation (2026). MySQL 8.0 Reference Manual: The InnoDB Storage Engine. Available at: <https://dev.mysql.com/doc/refman/8.0/en/innodb-storage-engine.html>